

Lab Report: Lab 4 - Microservices Decomposition & Communication

Contents

1. Cover Page	1
2. Abstract/Summary	1
3. Lab Specific Section: Microservices Decomposition & Communication	2
4. Conclusion & Reflection.....	3

1. Cover Page

- **Title:** Microservices Decomposition and Communication Strategy for OmniDash
 - **Course Info:** Software Architecture
 - **Student Details:** Nguyễn Văn Trường, 23010371
 - **Date:** May 4, 2026
-

2. Abstract/Summary

This lab marks the architectural transition of the OmniDash workspace automation platform from a monolithic Layered Architecture to a distributed Microservices Architecture. The primary objective was to logically decompose the system based on business capabilities, ensuring that each resulting microservice is independently deployable and strictly owns its respective data domain. We successfully mapped the core functionalities into distinct services (User, Task, Workspace, and Notification Services), defined robust RESTful Service Contracts to establish boundaries, and designed a high-level communication strategy utilizing both synchronous and asynchronous patterns. Furthermore, the system's macroscopic boundaries and external interactions were visualized using the C4 Model (Level 1: System Context). This theoretical decomposition is a critical prerequisite for building a scalable and fault-tolerant distributed system.

3. Lab Specific Section: [Microservices Decomposition & Communication](#)

This section documents the architectural artifacts resulting from the decomposition of the OmniDash system, detailing service boundaries, API contracts, and communication topologies.

3.1 [Decomposition by Business Capability](#)

To achieve high cohesion and loose coupling, the monolithic OmniDash backend was analyzed and segmented based on its primary business capabilities. Each identified capability was mapped to a dedicated microservice. A fundamental rule of this decomposition is the Database-per-Service pattern; no service is permitted to directly access another service's database.

Business Capability	Proposed Microservice	Data Owned (Entities)
User Identity & Auth	User Service	User Profiles, Passwords (hashed), JWT Configurations.
Workspace Automation	Workspace Service	Workspace Combos, Resource URLs, Tab Group Metadata.
Task Management	Task Service	Task Headers, Descriptions, Due Dates, Statuses.
System Alerts	Notification Service	Email Templates, Notification Logs, Delivery Statuses.

3.2 [Modeling Artifact: UML Use Case Diagram](#)

With the database boundaries strictly established, services must communicate via well-defined public interfaces known as Service Contracts. Below is the API specification for the Workspace Service, which handles the core automation logic of the platform.

Endpoint	HTTP Method	Description	Data Returned
/api/workspaces	GET	Retrieve a summarized list of workspaces owned by the user.	List of Workspace summary objects.
/api/workspaces/{id}	GET	Retrieve full details of a workspace, including its nested URL resources.	Detailed Workspace object.
/api/workspaces	POST	Create a new Workspace Combo containing multiple URLs.	The newly created Workspace object.
/api/workspaces/{id}	DELETE	Permanently remove a workspace and its associated resources.	Deletion confirmation message.

3.3 [Communication Strategy](#)

A robust distributed system requires a tailored communication strategy to balance performance,

reliability, and coupling. OmniDash employs a hybrid approach, utilizing both synchronous API calls and asynchronous message passing.

Interaction Scenario	Source → Target	Comm. Type	Architectural Rationale
Authentication Check	API Gateway \$→\$ User Service	Synchronous (HTTP/REST)	Immediate validation is required before granting access to protected routes.
Verify Workspace Existence	Task Service \$→\$ Workspace Service	Synchronous (HTTP/REST)	The Task Service needs immediate confirmation of the workspace's validity before saving a task.
Send Welcome Email	User Service \$→\$ Notification Service	Asynchronous (Message Queue)	Sending an email is a slow process that should not block the user registration workflow.

3.4 Model (Level 1: System Context)



4. Conclusion & Reflection

Lab 4 provided critical insights into the complexities of distributed system design. By decomposing the OmniDash monolith into distinct microservices (User, Task, Workspace, and Notification), we drastically improved the system's theoretical scalability and fault isolation.

Defining strict Service Contracts highlighted the importance of API design when databases are completely decoupled. Furthermore, mapping out the synchronous versus asynchronous communication strategies revealed how to prevent cascading failures—such as ensuring that an email service outage does not prevent users from registering. The C4 System Context diagram successfully established the system's macroscopic footprint. These design artifacts serve as the direct blueprint for the practical implementation of independent microservices and API Gateways in the subsequent labs.